

ReLePy: A Modular Reinforcement Learning Library with a Consistent, Configurable API

Reza Asadi¹

¹ Independent Researcher

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [↗](#)

Submitted: 25 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

ReLePy is a Python library that collects a broad range of reinforcement learning (RL) algorithms behind a single, consistent interface: every agent is built, trained, evaluated, saved and loaded through the same four methods (`fit`, `predict`, `evaluate`, `save/load`), regardless of whether it is a classical tabular method or a deep neural network trained with PyTorch. The library currently implements three tabular temporal-difference algorithms (Q-learning, SARSA, Expected SARSA), three value-based deep methods (DQN and its Double and Dueling variants), three on-policy policy-gradient methods (REINFORCE, A2C, PPO), and three off-policy actor-critic methods for continuous control (DDPG, TD3, SAC). Every algorithm has a dedicated, type-annotated, validated configuration object (a Python dataclass) that exposes its hyperparameters explicitly and can be serialized to and from JSON, which makes experiments easy to reproduce and to compare. ReLePy interoperates with any [Gymnasium](#) environment ([Towers et al., 2024](#)), supports vectorized (parallel) training for its on-policy algorithms, and stores trained models in a pickle-free archive format so that loading a saved agent never executes arbitrary code.

Statement of need

Reinforcement learning research and teaching both benefit from software that lets a user move between very different families of algorithms – tabular, value-based and policy-based – without learning a new interface each time. Existing mature libraries such as Stable-Baselines3 ([Raffin et al., 2021](#)) and CleanRL ([Huang et al., 2022](#)) are excellent resources, but they embody two different philosophies: Stable-Baselines3 favours a unified, production-style API but only implements deep, PyTorch-based algorithms, so a user who also wants classical tabular Q-learning or SARSA has to switch to a separate package; CleanRL favours single-file, easily readable reference implementations, at the cost of code duplication across algorithms and the absence of a shared, reusable base class. ReLePy is designed to sit between these two philosophies: it keeps one shared agent interface and one configuration pattern across all algorithm families, including tabular methods that require no deep learning framework at all, while still keeping each algorithm's implementation in its own short, readable module. This makes ReLePy suitable for teaching settings where students progress from tabular control to deep value-based and policy-gradient methods within one consistent codebase, and for practitioners who want to prototype quickly with an explicit, reproducible hyperparameter configuration.

ReLePy targets researchers, students and practitioners who need to (i) compare algorithms across the tabular/deep-value/policy-gradient/actor-critic spectrum within one API, (ii) keep hyperparameter configurations explicit, validated and serializable for reproducible experiments, and (iii) load saved models without exposing themselves to the arbitrary code execution risk that pickle-based formats carry, which is a documented pitfall of several existing RL and ML

42 model formats.

43 Comparison to related work

44 The table below summarizes how ReLePy relates to two widely used RL libraries along the axes
45 most relevant to the design goals above.

	ReLePy	Stable-Baselines3	CleanRL
Tabular algorithms (Q-learning, SARSA)	Yes	No	No
Deep value-based / policy-gradient / actor-critic	Yes	Yes	Yes
Shared base class across all algorithms	Yes	Yes (deep only)	No (single-file design)
Typed, validated, JSON-serializable configs	Yes	Partial (dict-based)	No (argparse flags)
Vectorized (parallel) environment training	Yes (on-policy)	Yes	Yes
Pickle-free saved model format	Yes	No	N/A

46 ReLePy's PPO implementation was benchmarked against Stable-Baselines3's PPO on the classic
47 control task CartPole-v1 with matched hyperparameters (learning rate, rollout length, batch
48 size, number of epochs, discount factor, GAE lambda, clip range) and an identical training
49 budget of 50,000 environment steps, averaged over three random seeds. Both implementations
50 reached the maximum possible return of 500 with zero variance across seeds, while ReLePy
51 completed training in approximately 44% of the wall-clock time of Stable-Baselines3 on the
52 same machine, indicating that the added flexibility of ReLePy's design does not come at the
53 cost of either final policy quality or training speed on this benchmark. The benchmark script
54 is included in the repository (examples/benchmark_vs_sb3.py) so that it can be reproduced
55 or extended to further tasks.

56 Design and implementation

57 Every ReLePy agent subclasses a common BaseAgent abstract base class that implements
58 the public fit/predict/evaluate/save/load interface and delegates the algorithm-specific
59 learning update to subclasses. Hyperparameters are represented as dataclass-based
60 configuration objects with a validate method that raises immediately on out-of-range or
61 inconsistent values, and which support serialization to and from JSON for recording exact
62 experimental settings. Training progress can be monitored and controlled with a small callback
63 system (evaluation on a held-out environment, checkpointing, and early stopping on a reward
64 threshold), modeled after similar mechanisms in other RL libraries. Saved models are written
65 as a ZIP archive containing JSON metadata and NumPy .npy arrays rather than a pickle file,
66 so that loading a model downloaded from an untrusted source cannot execute arbitrary code –
67 a property not shared by several existing RL and general machine-learning serialization formats.

68 Acknowledgements

69 We acknowledge the authors of Gymnasium, PyTorch and Stable-Baselines3, whose design
70 choices and documentation informed several aspects of this library.

References

- Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., & Araújo, J. G. M. (2022). CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. In *Journal of Machine Learning Research* (No. 274; Vol. 23, pp. 1–18).
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., & others. (2024). *Gymnasium: A standard interface for reinforcement learning environments*. <https://github.com/Farama-Foundation/Gymnasium>.

DRAFT